

PeopleHub

MERN User Management Application

Technical Assessment Submission

Repository	github.com/srinivasanb2004/User-Management-Application
Live Application	user-management-application-ruddy.vercel.app
Live API	user-management-application-4ryy.onrender.com
Health Check	/api/health

A secure, responsive full-stack application for managing team member records. Built with MongoDB, Express.js, React, and Node.js.

Project Overview

PeopleHub provides administrators with a protected workspace for maintaining a team directory. The application combines a RESTful Express API, MongoDB persistence through Mongoose, and a responsive React interface.

Assessment Requirements Completed

Area	Completed implementation
Backend	Create, list, retrieve by ID, update, and delete user REST endpoints.
Data model	Full name, email address, phone number, role, and automatic date-created timestamp.
Validation	Zod request validation, Mongoose constraints, client-side form validation, and duplicate-email handling.
Error handling	Clear API responses for invalid data, invalid IDs, duplicate emails, JSON errors, missing routes, and server failures.
Frontend	Responsive dashboard with create, list, edit, delete, loading states, empty states, and notifications.

Core Features

- Protected administrator dashboard for user management.
- User registration form with validation for full name, email, 10-digit phone number, and role.
- Search by name or email, with server-side pagination.
- Confirmation before deletion, responsive mobile actions, and success/error toast notifications.
- Date-created display, role labels, loading indicators, and empty-result states.

Bonus Tasks Completed

- Search users by name or email.
- Server-side pagination with page and limit query parameters.
- JWT-based authentication with 8-hour sessions.
- Role-based access control: only administrators can manage users.
- Deployment: Vercel frontend, Render Node.js API, and MongoDB Atlas database.

Technology Stack

Layer	Technology
Frontend	React 19, Vite, CSS, Lucide React
Backend	Node.js, Express.js, Helmet, CORS, rate limiting
Database	MongoDB Atlas, Mongoose
Validation and auth	Zod, bcryptjs, JSON Web Tokens
Testing	Node test runner, Supertest, mongodb-memory-server

How to Review the Live Application

1. Open the live frontend: <https://user-management-application-ruddy.vercel.app>
2. Sign in with the administrator credentials provided separately by the candidate.
3. Review the team directory and test adding, editing, searching, paginating, and deleting a user record.
4. Confirm backend availability at: <https://user-management-application-4ryy.onrender.com/api/health>

Administrator credentials are intentionally not included in this public document or repository.

Local Setup and Run Instructions

Prerequisites: Node.js 20+ and a MongoDB database. MongoDB Atlas can be used for local and deployed environments.

1. Clone the repository

```
git clone https://github.com/srinivasanb2004/User-Management-Application.git
cd User-Management-Application
```

2. Install dependencies

```
npm.cmd install
npm.cmd install --prefix server
npm.cmd install --prefix client
```

3. Create the backend environment file

```
Copy-Item server\.env.example server\.env
```

4. Configure server/.env

Set PORT, MONGODB_URI, JWT_SECRET (32+ characters), ADMIN_EMAIL, ADMIN_PASSWORD (12+ characters), and CLIENT_URL=http://localhost:5173. The first startup creates the initial administrator if it does not exist.

5. Start the full application

```
npm.cmd run dev
```

Open <http://localhost:5173> and sign in with the configured administrator account.

API Summary

Method	Endpoint	Purpose
POST	/api/auth/login	Administrator login and JWT issue
GET	/api/auth/me	Return current authenticated user
GET	/api/users	List users; supports search, page, and limit
POST	/api/users	Create a user
GET	/api/users/:id	Retrieve one user
PUT	/api/users/:id	Update a user
DELETE	/api/users/:id	Delete a user

Verification

- Backend integration tests cover authentication, validation, duplicate-email handling, CRUD operations, search, and administrator protections.
- The React production build completes successfully with Vite.
- The deployed API health endpoint returns a successful status response.